C# Questions

1.What is the difference between == and Equals() in C#?

2.Can you explain what a namespace is and how it is used in C#?

3.What are value types and reference types in C#? Give examples.

4.What is inheritance and how is it implemented in C#?

5. Explain the SOLID principles.

6.What are interfaces and abstract classes? How are they different?

7.What is encapsulation and how is it achieved in C#?

8.Explain the concept of method overloading and method overriding with examples.

9.What are generics in C# and why are they useful?

10.Explain the difference between Array and ArrayList in C#.

11.What is the difference between String and StringBuilder in C#?

12.What is the difference between "ref" and "out" parameters in

C#? When would you use each one?

13.Describe the concept of serialization and deserialization in C#. How do you implement it using attributes or interfaces?

14.What is LINQ and how is it used in C#?

15.What is a Dictionary in C# and how is it different from a List?

16.Explain the concept of a lambda expression and provide an example.

17.What are anonymous types in C#?

18.What are extension methods in C#? How do you create and use extension methods?

19.Describe the use of delegates and events in C#. Provide an example of defining and using delegates and events.

20.What is the purpose of the "using" statement in C#? How is it related to IDisposable?

22.What are nullable types in C#? How do you handle null values using nullable types?
23. What is the difference between struct and class?
24. Explain abstraction vs encapsulation with examples.
25. What is the difference between readonly and const?
26. What is the use of async and await?
27. What is a sealed class? When to use it?
28. What are extension methods?
29. What is garbage collection? What are the GC generations?
30. What are delegates? What is a multicast delegate?
31. What is LINQ? Explain Select, Where, GroupBy with examples.
32. Difference between IEnumerable, IQueryable, List<T> and ICollection<T>?
33. What is dependency injection? Why is it used?
34. What is the difference between var, dynamic, and object?
35. What is exception handling? Explain try-catch-finally.
36. What is polymorphism? Explain method overloading & overriding.

ASP.NET WebForms Interview Questions

Basic Concepts

1. **What is ASP.NET WebForms?**
    - **Answer**: ASP.NET WebForms is a web application framework developed by Microsoft to build dynamic web pages. It is part of the ASP.NET framework and allows developers to build applications with a rich user interface using a drag-and-drop, event-driven programming model.
2. **Explain the Page Life Cycle in ASP.NET WebForms.**
    - **Answer**: The page life cycle in ASP.NET WebForms consists of several stages: Page Request, Start, Initialization, Load, Postback Event Handling, Rendering, and Unload. Each stage has its own events like Page_Init, Page_Load, Page_PreRender, etc., which allow developers to execute code at specific points.
3. **What are ViewState, SessionState, and ApplicationState?**

- **Answer**:
    - **ViewState**: Maintains the state of controls on a web page across postbacks.
    - **SessionState**: Maintains user data across different pages within a web application.
    - **ApplicationState**: Maintains data that is shared across all users and sessions within the web application.

4. **What is the difference between Server.Transfer and Response.Redirect?**
    - **Answer**:
        - **Server.Transfer**: Transfers the request from one page to another on the server side without changing the URL in the browser, conserving server resources.

        - **Response.Redirect**: Redirects the client's browser to a new URL, making a round-trip to the server, changing the URL in the browser.

Controls and Events

5. **What is the difference between Web User Control and Web Custom Control?**
    - **Answer**:
        - **Web User Control**: Easy to create and use, designed using .ascx files, ideal for reuse within the same application.
        - **Web Custom Control**: More complex to create, compiled into DLLs, can be reused across multiple applications, and offers greater flexibility.
6. **Explain the concept of Postback in ASP.NET WebForms.**
    - **Answer**: Postback is the process where an ASP.NET WebForms page sends data back to the same page on the server for processing. This typically occurs when a user interacts with a control that has AutoPostBack set to true or triggers a server-side event.
7. **How can you maintain the state of a web page across postbacks?**
    - **Answer**: State can be maintained using ViewState, ControlState, Hidden Fields, Cookies, Query Strings, Session State, and Application State.

Data Handling

8. **What are the different data-bound controls in ASP.NET WebForms?**
    - **Answer**: Common data-bound controls include GridView, Repeater, DataList, ListView, and DetailsView.
9. **How do you bind data to a GridView control?**
    - **Answer**: Data can be bound to a GridView control using its DataSource property and calling the DataBind() method. For example:

```
GridView1.DataSource = myDataSource;
GridView1.DataBind();
```

10. **Explain the purpose of the SqlDataSource control.**
    - **Answer**: The SqlDataSource control provides a way to connect to a database and execute SQL queries directly from the markup. It can be used to select, update, delete, and insert data without writing code-behind for these operations.

Advanced Concepts

11. **What are HTTP Handlers and HTTP Modules?**
    - **Answer**:
      - **HTTP Handlers**: Components that handle HTTP requests. For example, .aspx pages are processed by a page handler.
      - **HTTP Modules**: Components that handle events during the request pipeline. They can inspect or modify the request and response.
12. **Explain the concept of Master Pages in ASP.NET WebForms.**
    - **Answer**: Master Pages allow you to define a common layout for multiple pages in a web application. A master page provides a template with placeholders for content that is overridden by content pages.
13. **How do you implement authentication and authorization in ASP.NET WebForms?**
    - **Answer**:
      - **Authentication**: Validates the identity of the user. Common methods include Forms Authentication, Windows Authentication, and custom authentication.

        - **Authorization**: Determines what an authenticated user is allowed to do. This is typically managed using roles and permissions.

Performance and Optimization

14. **What are some techniques to improve the performance of an ASP.NET WebForms application?**
    - **Answer**: Techniques include:
        - Caching (Output Caching, Data Caching)
        - Minimizing ViewState
        - Using asynchronous operations
        - Optimizing database queries
        - Using CDN for static resources
15. **How do you handle exceptions in ASP.NET WebForms?**
    - **Answer**: Exceptions can be handled using try-catch blocks in code, the Page_Error method, custom error pages configured in web.config, and global error handling in the Application_Error event in Global.asax.

Miscellaneous

16. **What is the purpose of the Global.asax file?**
    - **Answer**: The Global.asax file, also known as the ASP.NET application file, is used to handle application-level events such as Application_Start, Application_End, Session_Start, and Session_End.
17. **Explain the role of Web.config in an ASP.NET WebForms application.**
    - **Answer**: The Web.config file is an XML configuration file used to configure application settings, connection strings, authentication, authorization, custom error pages, and other settings.
18. **What are user controls and how do you create them?**

- **Answer**: User controls are reusable components defined in .ascx files. They can be created by adding a new Web User Control to the project and then using it in pages or other user controls.

19. **How do you manage state across multiple web servers in a web farm?**
    - **Answer**: State management in a web farm can be achieved using:
        - Out-of-process session state storage (State Server or SQL Server)
        - Distributed caching solutions like Redis or Memcached
        - Sticky sessions using load balancer configuration

20. **What is the purpose of the Runat="server" attribute?**
    - **Answer**: The Runat="server" attribute indicates that an HTML element should be processed on the server side. It allows server-side code to interact with the control.

ASP.NET Core & MVC

1) What are the key differences between ASP.NET Core MVC and ASP.NET MVC?
2) Explain the Middleware concept in ASP.NET Core. How does Middleware pipeline work?
3) What is Dependency Injection (DI) in ASP.NET Core? How do you register and use services using DI?
4) Describe the role of ConfigureServices and Configure methods in the Startup class of an ASP.NET Core application.
5) What is Razor Pages in ASP.NET Core? How is it different from MVC pattern?
6) How do you handle routing in ASP.NET Core MVC? Explain attribute routing and convention-based routing.
7) What are Tag Helpers in ASP.NET Core? Provide examples of built-in Tag Helpers and how to create custom Tag Helpers.
8) Explain the use of View Components and Partial Views in ASP.NET Core MVC. How do they differ from each other?
9) Describe the differences between TempData, ViewData, and ViewBag in ASP.NET Core MVC for passing data between controllers and views.
10) Explain the request processing pipeline in ASP.NET Core.
11) What is data annotation? Name 5 commonly used attributes.
12) What is ViewModel? Why do we use it?
13) What is Partial View? How is it different from Layout View?
14) What is Anti-forgery token? Why is it used?
15) What is ASP.NET Core Identity? How do you implement authentication and authorization using Identity in ASP.NET Core MVC?
16) Explain the concept of Entity Framework Core (EF Core) in ASP.NET Core MVC for database access. How do you perform CRUD operations using EF Core?

17) What is Middleware Authorization in ASP.NET Core? How do you implement role-based or policy-based authorization?

18) Describe the use of Areas in ASP.NET Core MVC for organizing controllers, views, and other resources in large applications.

19) How do you work with client-side libraries and frameworks (such as jQuery, Bootstrap, etc.) in an ASP.NET Core MVC application?

20) Explain the Model-View-Controller (MVC) pattern. How does it work in ASP.NET Core MVC?

21) How do you configure services in the Startup class?

22) Explain how action methods work in ASP.NET Core MVC. How do you return different types of responses?

23) What is Model Binding? Different types of Model Bindings?

24) What are View Components in ASP.NET Core MVC? How do they differ from Partial Views?

25) How do you implement asynchronous action methods in ASP.NET Core MVC?

26) How do you create and use a custom Tag Helper?

27) How do you handle exceptions in ASP.NET Core MVC? Explain the use of exception filters and middleware.

28) What is the difference between handling exceptions in middleware and in exception filters?

29) How do you implement logging in an ASP.NET Core MVC application?

30) Explain how to use the built-in logging framework in ASP.NET Core.

31) How do you unit test controllers in ASP.NET Core MVC?

32) What tools and frameworks do you use for testing ASP.NET Core MVC applications?

33) What are Action Filters? Types of filters?

34) How to implement Client-side & Server-side validation in MVC?

35) What are Partial Views & View Components? Difference?

36) What is Startup.cs & Program.cs role in .NET 6/7?

37) What is IActionResult vs ActionResult<T>?

38) What is Razor View Engine? How it works internally?

39) How session and cookies work in .NET Core MVC?

40) How to prevent CSRF attacks in MVC?

41) Explain Lazy Loading, Eager Loading, and Explicit Loading

42) How do you handle **transactions** in EF Core?

43) How do you improve EF Core performance? (Compiled queries, AsNoTracking, pagination).

44) Difference between **JWT** and **OAuth2**.

45) Explain **Claims-based Authorization**.

46) How do you prevent **CSRF** and **XSS attacks** in ASP.NET Core?

47) What are **Migrations** in EF Core? How do you create and apply them?

48) Explain **Code First**, **Database First**, and **Model First** approaches.

49) How do you configure **relationships** (One-to-Many, Many-to-Many) in EF Core?

50) Difference between **IApplicationBuilder.Use()**, **Run()**, and **Map()**

51) Explain **ConfigureServices()** vs **Configure()** methods.

52) Difference between **appsettings.json** and **appsettings.Development.json**.

53) Difference between IHttpActionResult and HttpResponseMessage

54) Explain the difference between IServiceCollection, IServiceProvider, and ConfigureServices

55) What is the difference between **Use**, **Run**, and **Next** in Middleware?

56) What is the difference between **AuthorizationFilter**, **ActionFilter**, and **ExceptionFilter**?

57) What is the difference between **Cookie-based Authentication** and **Token-based Authentication**?

58) How can you improve **performance** in ASP.NET Core applications?

59) What is **Asynchronous Programming** in ASP.NET Core? How does it

improve performance?

60) How do you handle **global exception logging** in ASP.NET Core?

61) How do you log to **files, database, or cloud services** in ASP.NET Core?

62) What steps would you take to **troubleshoot a slow ASP.NET Core application**?

63) How do you handle **database concurrency** in EF Core?

64) How do you secure an ASP.NET Core application against **SQL Injection & XSS attacks**?

65)

66) Difference between JsonResult, ViewResult, PartialViewResult and ActionResult

67) How do you handle form submissions and validation in ASP.NET Core MVC? Describe the use of model validation attributes.

WEB API Questions

1) What is ASP.NET Core Web API, and how does it differ from traditional ASP.NET MVC?
2) Explain the concept of RESTful API and how ASP.NET Core Web API facilitates building RESTful services.
3) What are the different HTTP methods (verbs) supported by ASP.NET Core Web API, and what are their purposes?
4) Describe the use of Model Binding in ASP.NET Core Web API for parameter binding and validation.
5) How do you handle routing in ASP.NET Core Web API? Explain attribute routing and convention-based routing.
6) What is content negotiation in ASP.NET Core Web API? How do you return different types of content based on client requests?
7) Explain the use of ActionResult<T> in ASP.NET Core Web API for returning data along with HTTP status codes.
8) How do you return **custom HTTP status codes** from Web API?
9) What are **JWT tokens** and how are they used in Web API?
10) What are filters in ASP.NET Core Web API, and how do you use them for cross- cutting concerns like logging, validation, and authorization?
11) How do you handle exception handling in ASP.NET Core Web API? Describe the use of middleware and exception filters.
12) Describe the concept of versioning in ASP.NET Core Web API. How do you implement API versioning to support backward compatibility?
13) Explain the use of dependency injection (DI) in ASP.NET Core Web API. How do you register and use services using DI?
14) What is CORS (Cross-Origin Resource Sharing), and how do you configure CORS policies in ASP.NET Core Web API to allow or restrict access from different domains?
15) What are the different ways of testing ASP.NET Core Web API? Describe unit testing, integration testing, and end-to-end testing

approaches

16) What are **DTOs (Data Transfer Objects)**? Why do we use them?

17) How do you implement authentication and authorization in ASP.NET Core Web API? Describe the use of JWT (JSON Web Tokens) and OAuth2

18) How do you secure a Web API using **Bearer Token**?

19) How do you implement **global exception handling** in ASP.NET Core Web API?

20) Difference between **JWT** and **OAuth2**.

21) How do you implement **pagination** in Web API?

22) How do you implement **caching** in Web API (in-memory, distributed, response caching)?

23) Your API needs versioning (v1, v2). How do you design it?

24) A client uploads multiple files via Web API. How do you implement validations (size, type)?

25) How do you **log API requests and responses**?

26) How do you handle **circular references** in JSON serialization?

27) How would you design an API that supports **multi-tenant applications**?

28) If two APIs need to **communicate with each other**, how would you implement it?

29) How do you implement **soft delete** in EF Core?

30) If your Web API is **slow**, how would you diagnose and fix it?

31) Explain FromBody, FromQuery, FromRoute, FromForm, FromHeader.

32) What is JWT Authentication? How it works?

33) What is HATEOAS? Is it used in modern APIs?

34) How to secure Web API using JWT + Role-based authorization?

35) How to consume another API inside your API? HttpClient best practices?

36) What rate limiting is? How to implement it in .NET?

37) What is API Throttling?

38) How to implement File Upload & Download API?

39) What is REST? What are REST principles?

40) How do you secure APIs?

41) Explain IHostedService and background tasks.

42) What is caching? Types of caching in API?

43) What is the difference between ILogger, ILoggerFactory, and ILogger<T>?
44) Difference between REST API and SOAP API?
45) How do you return **large files or streaming data** in Web API?

Angular Questions:

1.What are the main features introduced in Angular 7+?

2.Explain the difference between Angular and AngularJS.

3.What is TypeScript, and why is it used in Angular?

4.Explain the purpose of Angular CLI.

5.What is the difference between ngIf, ngFor, and ngSwitch?

6.How do you create a new Angular project using CLI?

7.What are Components, Directives, and Services in Angular?

8.Explain Interpolation and Property Binding in Angular.

9.What is Two-way Data Binding? How does it work?

10.What are Event Binding and Attribute Binding?

11.Explain the difference between @Input() and @Output() decorators.

12.How can you pass data between parent and child components?

13.What are Structural Directives and Attribute Directives?

14.What is the difference between ngClass and ngStyle?

15.What are Pipes in Angular? Explain built-in pipes and how to create a custom pipe.

16.What is a Service in Angular? Why is it used?

17.Explain the concept of Dependency Injection (DI) in Angular.

18.How do you create and use a service in Angular?

19.How do you configure routing in Angular?

20.What is the difference between RouterLink and href?

21.How can you pass parameters in Angular routes?

22.What are Route Guards? Name different types of route guards.

23.Explain the difference between Template-driven and Reactive Forms.

24.How do you perform form validation in Angular?

25.How can you dynamically add form fields in a Reactive Form?

26.How do you make an HTTP GET and POST request in Angular?

27.What is the difference between HttpClientModule and HttpModule?

28.How do you handle errors in HTTP requests in Angular?

29.What is RxJS, and why is it used in Angular?

30.Explain Observables and Promises. Which one is better in Angular?

31.What is BehaviorSubject in RxJS?

32.How do you improve performance in an Angular application?

33.What is Lazy Loading in Angular, and why is it used?

34.What is Change Detection in Angular?

35.Explain the difference between OnPush Change Detection Strategy and Default Change Detection.

36.What are Angular Lifecycle Hooks? Explain each one briefly.

37.What is Angular Universal?

38.What are Web Workers, and how do they help in Angular applications?

39.How do you implement Authentication & Authorization in Angular?

◈ Scenario-Based ASP.NET Core & Web API Questions

1) Your API needs versioning (v1, v2). How do you design it?
2) You need to return custom error messages in a consistent format for all API endpoints. How do you do it?
3) An API is being called very frequently with the same data. How do you implement caching?
4) One of your APIs needs authentication with JWT. How do you implement it?
5) You need to call another API inside your Web API (chained calls). How do you design this?
6) How will you handle exceptions globally in your Web API?
7) A client complains that your API is slow because of returning too much data. How do you optimize it?
8) How do you handle transactions if you are saving data into multiple tables?
9) You need to secure an API for internal apps only. How do you restrict external users?
10) You have two roles (Admin, User). Admin can access everything; User can only read. How do you implement this?
11) You need to prevent CSRF attacks in MVC. How will you do it?
12) How will you protect your Web API endpoints from brute force attacks?
13) You need to log which user accessed which endpoint. How will you implement this?
14) Your app works locally but fails in production with connection string issues. How will you troubleshoot?
15) You deployed your API, but Angular frontend gets CORS errors. How do you fix it?

16) If two teams are working on different modules in the same solution, how will you manage branching/merging in Git?

17) Your application needs to support multiple languages (English, Hindi, Marathi). How do you design it?

18) One API request takes longer than expected. How do you debug and improve it?

19) You need to build an API for an Employee Management System (CRUD). How will you design it with DTOs?

20) A mobile app consumes your Web API. They need different response formats (JSON, XML). How do you support both?

21) You need to return custom error messages in a consistent format for all API endpoints. How do you do it?

22) An API is being called very frequently with the same data. How do you implement caching?

23) One of your APIs needs authentication with JWT. How do you implement it?

24) You need to call another API inside your Web API (chained calls). How do you design this?

25) How will you handle exceptions globally in your Web API?

26) A client complains that your API is slow because of returning too much data. How do you optimize it?

27) How do you handle transactions if you are saving data into multiple tables?

28) You have two APIs — OrderAPI and PaymentAPI. When an order is placed, PaymentAPI must be called automatically. How would you handle this?

29) You deployed your Web API, but clients report intermittent failures. How do you debug and monitor it in production?

30) Your public API is getting too many requests from a single client, slowing down others. How would you handle it?

31) Your API endpoint returns 1 lakh employee records and the

response is very slow. How do you optimize it?

32) Your company has an Employee API in production, but now you need to add extra fields in the response without breaking existing clients. How do you handle this?